OPEN SOURCE MIT-0

Agentic Coding Harness & Benchmarks

Cut agentic-coding **token spend** by choosing the harness and model from a measured **cost/quality frontier** -- on Amazon Bedrock or self-hosted.

CLAUDE CODE, PI, OMP & KIRO-CLI AMAZON BEDROCK SELF-HOSTED VLLM LLM-AS-JUDGE

Practitioner + executive briefing

What you'll get out of this

A buying guide

A measured cost/quality frontier -- pick the model on evidence, not reputation.

The overlooked lever

The agent matters as much as the model -- the same model lands in two different places under two harnesses.

Your hosting, your call

One measurement across Amazon Bedrock and self-hosted GPUs -- decide on a level footing.

And the skill that reads the frontier for you: swe-router names the cheapest model that clears the bar for the task in front of you.

What we'll cover

01 **The problem: agentic-coding token spend**

02 **What we built, and what it measures**

03 **Run this in your organization**

04 **Methodology: how cost is measured**

A quick read for executives, with practitioner and methodology detail as you go deeper.

01

The problem: token spend

Every developer and every task adds to the bill. The two biggest levers -- which harness, which model -- get chosen on gut feel.

Agentic coding burns tokens at scale

Long-horizon, not one-shot

A task is tens to hundreds of LLM turns (~50-250, up to 300+), running 10 minutes to over an hour -- not a single prompt/response.

Prefill-heavy shape

Each turn replays a growing transcript as fresh input and emits a tiny edit. Real input:output runs **~150:1 to ~660:1** -- not the ~3:1 generic pricing assumes.

Cost hides in the choices

How many turns, how much context re-read, which harness, which model -- these swing the bill several-fold for similar quality.

Organizations are already at -- or heading toward -- trillions of tokens per month, and coding agents drive a large and growing share of it. That is the bill this repo helps control.

Public leaderboards can be saturated, and generic token pricing does not reflect this workload -- you need numbers on work that looks like yours.

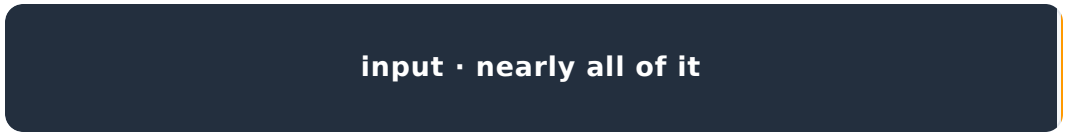
Why it's different

Coding spend doesn't look like app traffic

App traffic
single-shot · small / volume models

~3:1

Agentic coding
~50–250 turns · 10 min–1 hr+ · frontier models



~150:1 → 660:1

input output Same bar width - watch the output slice all but vanish for coding. That's where the cost is.

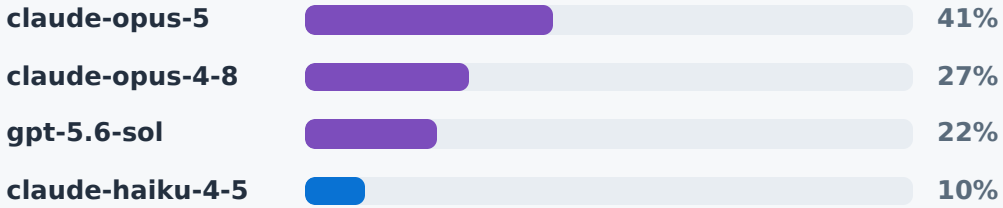
Generic pricing and leaderboards don't tell you the bill. Only benchmarking on real tasks does.

The mix

Two organizations, the same coding work

ORG A · FRONTIER-ONLY

Everything runs on a premium model



~90% of volume on premium. Nothing routes down. The bill scales with headcount × frontier price.

ORG B · GOVERNED MIX

Premium where it counts, open-weight for the rest

~70% of volume on open-weight models at a fraction of the per-token price - premium spread across two vendors and reserved for work that needs it.

Premium / frontier · Small proprietary · Open-weight, self-hosted

Same developers, same tasks - **very different bills**. The gap isn't discipline; it's whether anyone measured which tasks actually need a frontier model.

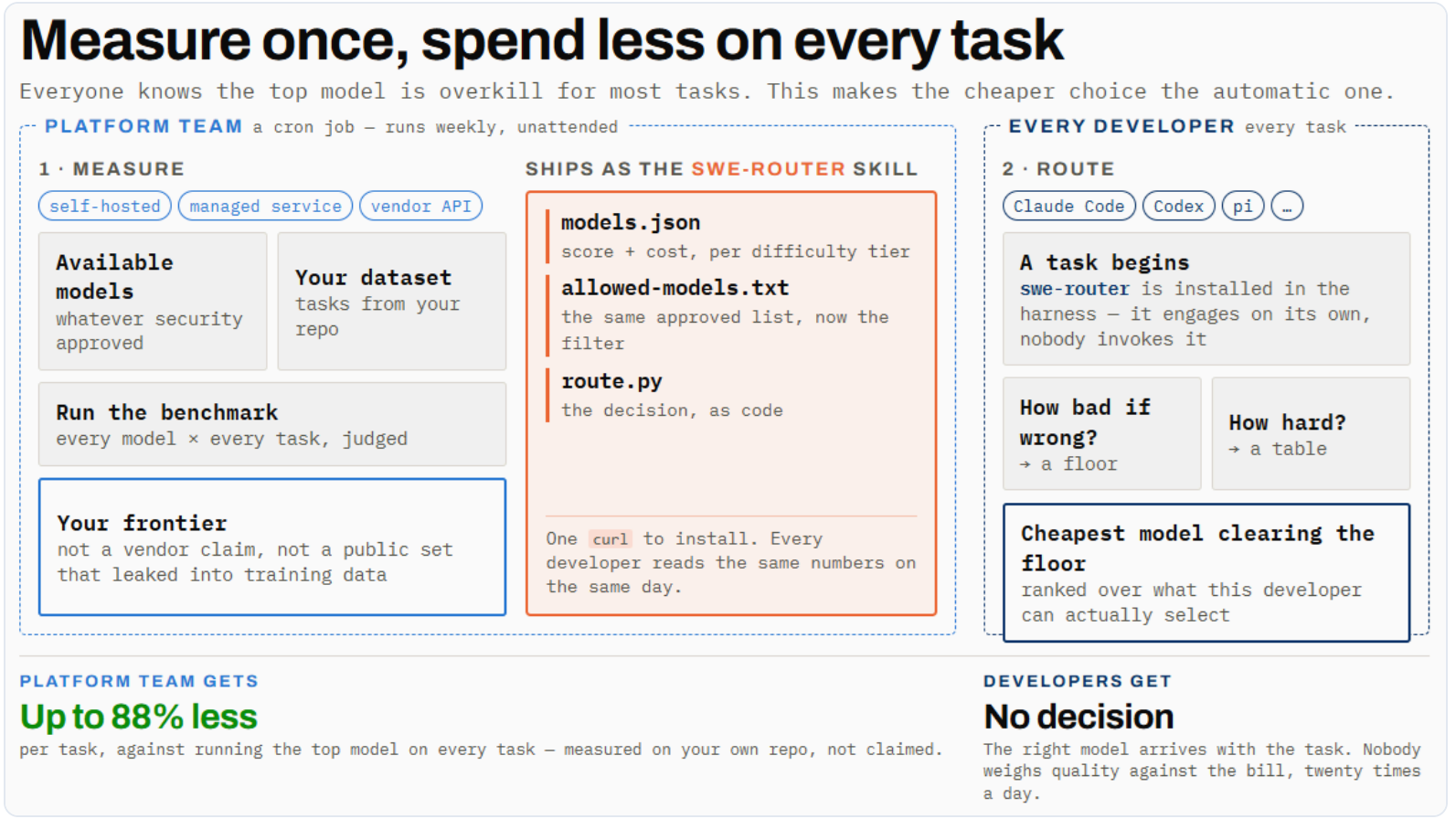
Bars show **share of token volume**, not share of cost - the cost skew is steeper still. Illustrative mixes, not a specific organization; confirm the shape against your own gateway telemetry.

02

The approach & the headline

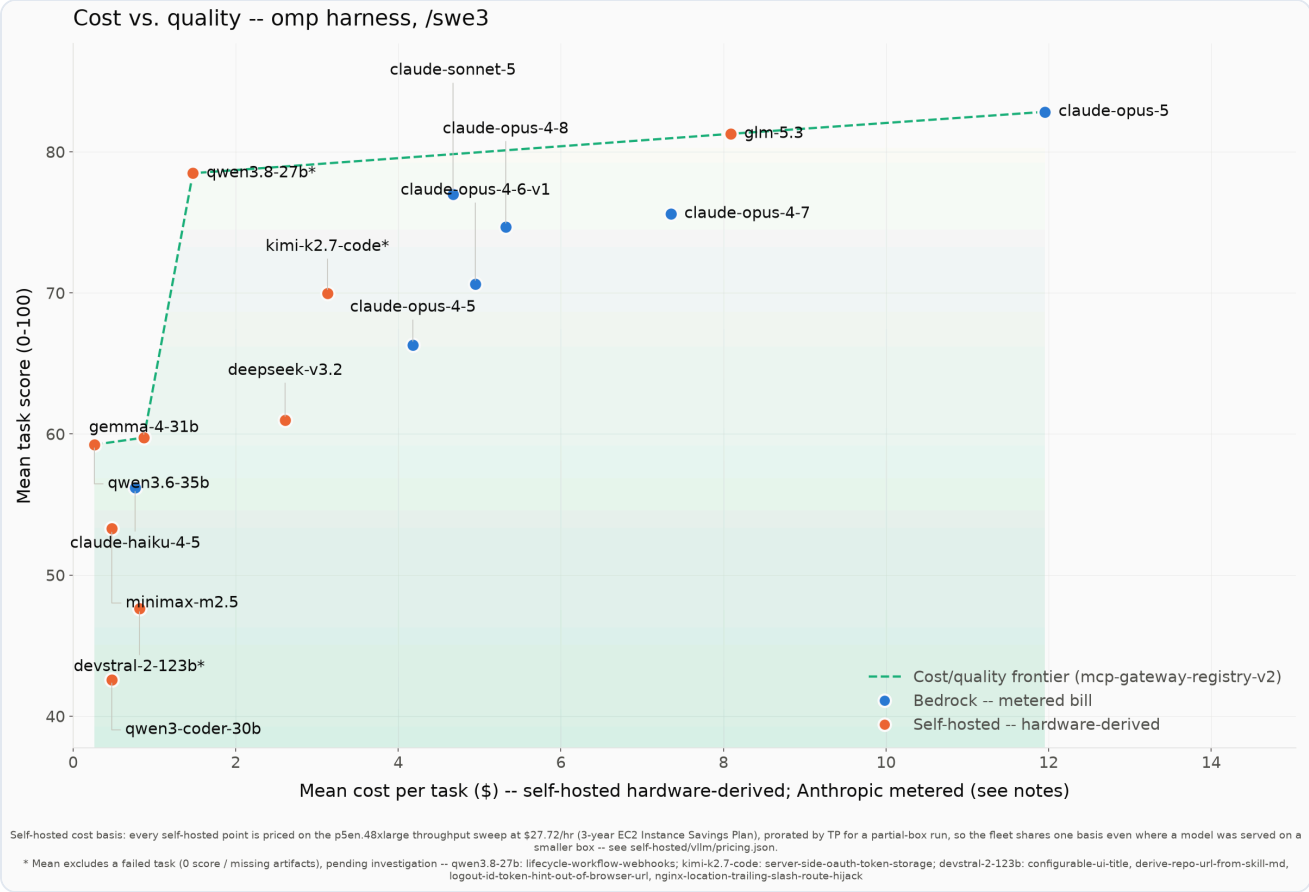
Measure quality and cost on real tasks, across harnesses and hosting paths, then let the Pareto frontier guide the choice.

Measure once, spend less on every task



The platform team owns the left half and runs it unattended. The developer never leaves their editor: the skill is already installed, engages on its own, and prints a recommendation before any work starts.

The cost/quality frontier (omp, /swe3)



16 models, 21 real tasks on `mcp-gateway-registry-v2`. x = mean cost/task, y = mean score. The dashed line is the non-dominated frontier. Anthropic points are metered Bedrock bills; self-hosted points are hardware-derived (different bases -- compare within a hosting basis).

Where models are strong or weak

The judge scores four criteria (complete, correct, specific, risk-aware) across the artifacts. Every model dips hardest on **implementation** and **correctness** -- landing working code is the hard part.

swe-router, in use

[placeholder]

Screen recording: a developer starts a task, the skill engages on its own, prints its recommendation, and the developer switches model.

Replace with the terminal capture (animated GIF or MP4) before presenting.

ADOPTION

03

Run this in your organization

How the measurement becomes a habit, where the decision belongs, and what it takes to hand your developers the result.

Scan to explore

The full harness, benchmarks, results, and methodology -- open source under MIT-0.

github.com/aarora79/agentic-coding-harness-benchmarks

Scan for the repository

Run it on your own code

Nothing here is specific to the example repository. Point the harness at your repos, run your model list through it, and the same generators build the frontier for **your** code. Then install **swe-router** so your developers spend that measurement without looking anything up.

1. Point at a repo

Any GitHub URL + a task description. The example repo is the example, not the contract.

2. Pick a path

Anthropic on Bedrock, open-weight on Bedrock (LiteLLM), or self-hosted vLLM.

3. Read your frontier

Score with the judge, plot cost against quality, then ship the result as the skill.

GitHub Repository **Install swe-router**

Why it matters

Make it easy to do the right thing

Continuous benchmarking - like evals - is desirable, not a burden: new models launch every week, so the frontier keeps moving.

The frontier only exists when all three come together - which is why **no single model provider can hand it to you**. You measure it on your own tasks.

NOW · RUN IT AS A HABIT

A framework + open-source implementation

The framework *and* the code to benchmark model × harness × hosting on your own real tasks. Re-run it regularly; hand your developers the result as an approved model/harness guideline.

NEXT · SHIFT LEFT SHIPS TODAY

An "swe-router" skill, inside any harness

Drops the frontier into your agentic-coding harness - whichever one you use - so the right model and harness are chosen automatically. Your developers stop carrying that decision.

Framework & implementation: agentic-coding-harness-benchmarks · swe-router: docs/vision.md · issue #161

Where the decision lives

Routing belongs where the context lives

APP TRAFFIC · GATEWAY-SIDE

Stateless - one hop, no repo, no follow-up.

AGENTIC CODING · SHIFTED LEFT INTO THE IDE

Stateful - judgement over many turns.

Same word, two different jobs. At the gateway it is **routing** - a classification made on the prompt alone, which is right for application traffic. For coding the context *is* the repository, so the choice is **decisioning**: a judgement the harness can only form after it has looked around and asked. In effect the decision **shifts left - out of the gateway and into the IDE**, where the agentic coding harness already runs.

Follows from the input-heavy profile earlier: ~50-250 turns replaying a growing transcript, not a single self-contained request. Gateways still own the things that *are* stateless - rate limits, spend caps, policy.

Where it's going

Take the model choice off the developer

Task + repo → tier → frontier lookup → model · harness · effort. A `swe-router` skill triages the task, takes the **cheapest model on the frontier that clears the bar**, and escalates a tier if the result falls short.

OPTION 1 · START HERE · ADVISE

Tell the developer, loudly

"For this task, switch to **this model at this reasoning effort**." The developer keeps their harness and can override it. No trust barrier - and every accepted call is evidence the routing works.

OPTION 2 · THEN · EXECUTE NEXT

Run it headless, hand back the result

The skill launches a **headless instance** of the chosen harness, at the chosen model and thinking effort, and completes the task. No manual switch, no decision left on the developer.

Sequence it: option 1, then option 2. Advisory mode earns the trust; move to headless once developers stop second-guessing the pick. Same frontier underneath - only the amount of automation changes.

Design: docs/vision.md · issue #161 - `swe-router`. Tiers: frontier / workhorse / budget, bounded escalation, every decision logged to `routing.json`.

METHODOLOGY DEEP-DIVE

04

Methodology

How a run works end to end, how cost is derived (GPU-seconds × $/second), and where models are strong or weak.

Two benchmarks, combined over real tasks

Quality

How well a model does a real coding task -- scored 0-100 by an independent LLM judge (**codex exec, GPT-5.6-sol**, high reasoning effort) against a fixed rubric.

Throughput → cost

Tokens/sec a self-hosted model sustains on a GPU, turning the instance's hourly price into a hardware-derived cost per task.

= Pareto frontier

Plot cost vs quality: the models where nothing else is both better and cheaper. That is the buying guide.

Crossing **harnesses** (Claude Code, pi, omp, kiro-cli) with **models** across **three hosting paths** gives real optionality -- the same model can be pricier under one agent yet cheaper and faster under another.

What is a Pareto frontier?

The set of models where no other model is both higher-scoring AND cheaper. Those are the only models worth considering. Everything else is *dominated* -- some frontier model beats it on both axes, so there is never a reason to pick it.

On the frontier

Non-dominated: to do better you must pay more, and to pay less you must accept a lower score. The real trade-off curve.

Dominated (skip it)

Example from our data: **kimi-k2.7-code (69.98, $3.13/task)** is beaten by **qwen3.8-27b (78.48, $1.47/task)** -- higher score *and* lower cost (both self-hosted). kimi-k2.7-code is off the frontier.

How to read it

Pick the quality your task needs (y-axis), then take the **leftmost model on the frontier** at that level -- the cheapest way to buy that quality.

Cost bases differ (metered Bedrock vs hardware-derived self-hosted), so we also draw the frontier within each hosting basis.

From long-horizon task to a cost/quality point

1 · Long-horizon task

Point the agent at a real repo + issue. It drives an open-ended tool-use loop over **~50-250 turns** (up to 300+), ~10 min to over an hour per task.

2 · Prefill-heavy tokens

Each turn replays the growing transcript as fresh input and emits a tiny edit -- **~150:1 to ~660:1** input:output. Measured, not assumed.

3 · Six artifacts

Lands **github-issue, LLD, review, testing** + the implemented change (**patch.diff, implementation.md**) on disk.

4 · Judge the quality

An independent judge (**codex exec, GPT-5.6-sol**, high effort) scores each artifact 0-100 on **completeness, correctness, specificity, risk-awareness** against a fixed rubric, read-only against a fresh clone.

5 · Marry with throughput

Multiply each run's real tokens by the model's measured tokens/sec on its GPU to get a **hardware-derived cost per task** -- quality × cost on the same real work.

= One frontier point

Every model becomes a (cost, quality) point; the non-dominated set is the frontier a buyer chooses from.

Two non-comparable cost bases

Metered (Bedrock)

A hosted API's real per-token bill, summed over the run. Benefits from prompt caching -- cache-read tokens billed at ~10%.

Hardware-derived (self-hosted)

A rented GPU has no per-token bill. Cost = **GPU-seconds × $/second** = (tokens ÷ measured throughput) × instance $/hr. Not wall-clock × $/hr -- that would charge idle agent-thinking time.

Compare within a hosting basis. Cross-hosting dollars are an order-of-magnitude result, not an exact tie -- each side credits prompt caching on its own terms.

Full cost methodology →

What lowers self-hosted cost

Concurrency, up to the KV wall

Amortize the fixed $/hr across more sessions -- until the KV cache saturates. On a big model that wall comes early (GLM-5.2 saturates at ~c=5 on 8×H200).

Model-to-box fit

A small-active MoE that fits on half the box serves far more concurrent sessions per dollar than a model that fills it.

Reserved / committed pricing

The leaderboard prices both GPU families at their 3-year commitment rate (p5en.48xlarge $27.72/hr, g6e.12xlarge $4.53/hr, in pricing.json). On-demand is ~2.3x higher and a 1-year plan sits between -- each shifts the frontier, and cost is linear in $/hr.

Caching is **already baked into the throughput** (98-99% prefix-cache hit rate raises tokens/sec) -- so it shows up as a lower rate, not a separate discount. The lever is **KV headroom**, which trades against context window and therefore accuracy: no free lunch.

HELD BACK

05

Backup

Model-by-model detail, for the questions that come after the talk.

What the frontier tells a buyer

82.8

claude-opus-5

Top quality, $11.95/task -- the premium tier for business-critical work.

81.3

glm-5.3

Best open-weight model at $8.09/task; the self-hosted quality anchor.

78.5

qwen3.8-27b

Within 4.4 points of the top model at $1.47/task -- an eighth of its cost.

A 27B open-weight model sits 4.4 points behind the best model at an eighth of the cost. Which model you pick moves the bill more than anything else you control.

Pick by tier (omp, /swe3, 21 tasks)

Premium `claude-opus-5`	Business-critical / high-stakes changes. Highest quality (82.83) at $11.95/task, finishing 21/21; wrong designs are expensive here.
Premium open-weight `glm-5.3`	Best self-hosted quality (81.27, $8.09/task), 1.6 points off the top model. Standardize on it if you self-host your frontier tier.
Reliable workhorse `qwen3.8-27b`	Bulk of day-to-day self-hosted coding: 78.48 at $1.47/task. It finished 20 of 21, so check the one it drops.
Most cost-effective `qwen3.6-35b`	Boilerplate, small fixes, scaffolding you'll review anyway: 59.24 at $0.26/task, finishing 21/21.

A cheap model that sits off the frontier, or that does not finish the task, is no bargain.

Does the harness matter? Yes.

Same model, same 21 v2 tasks	omp score	omp $/task	pi score	pi $/task
claude-opus-5	82.83	$11.95	77.62	$5.83
claude-sonnet-5	76.97	$4.67	72.53	$2.31
claude-haiku-4-5	56.18	$0.76	57.26	$0.54

The harness moves both axes

omp buys 5.2 points on opus-5 and 4.4 on sonnet-5, and charges about twice as much for them. On haiku it loses on both: pi scores 1.1 higher for 30% less.

So pick it per model

The same model lands in two different places under two harnesses. A harness is a cost lever the way a model is, and neither choice is safe to make once and forget.

The three models run under both harnesses on the v2 set. Both are metered Bedrock bills, so the dollars compare directly. Sources: harness-omp-swe3.md and results-swe3-v2.md.

If you put both hosting paths on one axis

Combined frontier (omp /swe3, v2)	Hosting	Score	$/task	Done
qwen3.6-35b	self-hosted	59.24	$0.26	21/21
gemma-4-31b	self-hosted	59.74	$0.87	21/21
qwen3.8-27b	self-hosted	78.48	$1.47	20/21
glm-5.3	self-hosted	81.27	$8.09	21/21
claude-opus-5	Bedrock	82.83	$11.95	21/21

Four of the five models on the combined frontier are open-weight -- glm-5.3 (81.27) sits 1.6 points under claude-opus-5 at two thirds the cost, and qwen3.8-27b reaches 78.48 for $1.47. Only the top of the frontier is a hosted API.

Directional only: metered Bedrock bills and hardware-derived self-hosted costs are not comparable as raw dollars. Reserved/committed GPU pricing pulls more open-weight models onto the frontier (see the methodology section).